Android Studio – All 10 Mobile Application Development Practicals

Windows 10/11

Java + XML

I will prepare the complete Android Studio instructions for all 10 practicals based on your uploaded MCA Semester III Mobile Application Development Lab Manual.

For every practical, I will provide:

1. Exact Android Studio click-by-click instructions.
2. Project name and project creation settings.
3. Exact Java and XML folder locations.
4. Which files to create.
5. Which existing files to open and modify.
6. What to select and where to save.
7. How to run the application.
8. How to test and verify the expected output.

No aims, theory, Java code, XML code, or viva questions. Only practical implementation steps.

Project plan for all 10 practicals

Practical	Project name	Action
1. Hello World	HelloWorld	Create new project
2. Activity Life Cycle	LifecycleDemo	Create new project
3. Registration Form	RegistrationForm	Create new project
4. Calculator UI	CalculatorUI	Create new project
5. Resources and Toast	RegistrationForm	Extend Practical 3
6. Two Fragments	FragmentDemo	Create new project
7. Fragment Communication	FragmentDemo	Extend Practical 6
8. Intents	IntentDemo	Create new project
9. SQLite Create and Insert	SQLiteDemo	Create new project
10. SQLite Retrieve and ListView	SQLiteDemo	Extend Practical 9

Important: Do not create a new project for Practical 5, 7 or 10. Continue using the respective earlier projects.

Practical 1: Android Studio Setup and Hello World

Project: HelloWorld

Step 1: Create the project

1. Open Android Studio.
2. Click New Project.

3. Select Phone and Tablet.
4. Select Empty Views Activity.
5. Click Next.
6. Enter project name: HelloWorld.
7. Enter package name: com.example.lab.
8. Select Language: Java.
9. Select Minimum SDK: API 23 or higher.
10. Choose the save location on your Windows computer.
11. Click Finish.
12. Wait until Gradle synchronization finishes.

Step 2: Open the Java folder

1. Open the Project panel on the left.
2. Change the dropdown to Android view.
3. Expand app.
4. Expand java.
5. Expand com.example.lab.
6. Open MainActivity.java.
7. This is your main Java file.

Location:

app/src/main/java/com/example/lab/MainActivity.java

Step 3: Open the XML folder

1. Expand app.
2. Expand res.
3. Expand layout.
4. Open activity_main.xml.
5. Select Code view in the layout editor.

Location:

app/src/main/res/layout/activity_main.xml

Step 4: Check AndroidManifest.xml

1. Expand app.
2. Open manifests.
3. Open AndroidManifest.xml.
4. Check that MainActivity is registered as the launcher Activity.
5. Do not remove the automatically generated application configuration.

Location:

app/src/main/AndroidManifest.xml

Step 5: Save and run

1. Save your changes using Ctrl + S.
2. Open Tools → Device Manager.
3. Click Create Virtual Device.
4. Select Phone.
5. Select a Pixel device.
6. Click Next.
7. Select an Android system image.
8. Download it if necessary.
9. Click Finish.
10. Start the emulator using the Play icon.
11. Return to Android Studio.
12. Select your emulator in the device dropdown.
13. Click the green Run button.
14. Wait for the application to install.

Step 6: Test

1. Check whether the application opens.
2. Verify that Hello World! appears in the centre of the screen.
3. Capture a screenshot using Win + Shift + S.
4. Save it in your Practical 1 screenshots folder.

Expected output

Hello World! displayed on the Android emulator.

Practical 2: Activity Life Cycle with Logcat

Project: LifecycleDemo

Step 1: Create a new project

1. Open Android Studio.
2. Click File → New → New Project.
3. Select Phone and Tablet.
4. Select Empty Views Activity.
5. Click Next.
6. Enter project name: LifecycleDemo.
7. Enter package name: com.example.lab.

8. Select Java.
9. Select Minimum SDK API 23 or higher.
10. Choose your project save location.
11. Click Finish.
12. Wait for Gradle synchronization.

Step 2: Open MainActivity.java

1. Open the Project panel.
2. Expand app.
3. Expand java.
4. Expand com.example.lab.
5. Double-click MainActivity.java.
6. Keep this file open for the Activity Life Cycle implementation.

Location:

app/src/main/java/com/example/lab/MainActivity.java

Step 3: Open activity_main.xml

1. Expand app.
2. Expand res.
3. Expand layout.
4. Open activity_main.xml.
5. Select Code view.
6. Keep this file open for the application screen.

Location:

app/src/main/res/layout/activity_main.xml

Step 4: Check the application manifest

1. Expand manifests.
2. Open AndroidManifest.xml.
3. Check that MainActivity is the launcher Activity.
4. Keep the existing configuration.

Location:

app/src/main/AndroidManifest.xml

Step 5: Save and build

1. Save all modified files.
2. Click Build in the top menu.
3. Select Make Project.

4. Wait for the build to finish.
5. Check that no red error appears.

Step 6: Run the application

1. Open Device Manager.
2. Start your Pixel emulator.
3. Select LifecycleDemo as the run configuration.
4. Select the emulator.
5. Click Run.
6. Wait for the application to open.

Step 7: Open Logcat

1. Find the Logcat tool window at the bottom of Android Studio.
2. Click Logcat.
3. Select your running emulator.
4. Select the LifecycleDemo process.
5. Find the Logcat search field.
6. Enter the filter LIFECYCLE.
7. Observe the Activity Life Cycle messages.

Step 8: Test the Activity Life Cycle

Perform the following tests separately:

Test A – Application launch

1. Close the application.
2. Open LifecycleDemo again.
3. Observe the first three Activity callbacks in Logcat.

Test B – Home button

1. Keep the application running.
2. Press the emulator Home button.
3. Check the Logcat messages.

Test C – Reopen application

1. Open the application again from the emulator.
2. Check the Logcat messages.

Test D – Screen rotation

1. Keep the application running.
2. Use the emulator Rotate button or keyboard rotation shortcut.
3. Observe the Activity recreation messages.

Test E – Back button

1. Open the application.
2. Press Back.
3. Observe the Activity destruction messages.

Step 9: Capture screenshots

Save two screenshots:

- LifecycleDemo running on the emulator.
- Logcat displaying the LIFECYCLE filter and callback messages.

Expected output

Activity Life Cycle messages visible in Logcat.

Practical 3: User Registration Form using Linear Layout

Project: RegistrationForm

Step 1: Create the project

1. Open Android Studio.
2. Select File → New → New Project.
3. Select Phone and Tablet.
4. Select Empty Views Activity.
5. Click Next.
6. Enter project name: RegistrationForm.
7. Enter package name: com.example.lab.
8. Select Java.
9. Select Minimum SDK API 23 or higher.
10. Select the save location.
11. Click Finish.
12. Wait for Gradle synchronization.

Step 2: Open the Java file

1. Expand app.
2. Expand java.
3. Expand com.example.lab.
4. Open MainActivity.java.

Location:

app/src/main/java/com/example/lab/MainActivity.java

Keep this file ready for the registration form functionality.

Step 3: Open the XML layout

1. Expand app.
2. Expand res.
3. Expand layout.
4. Open activity_main.xml.
5. Select Code view.

Location:

app/src/main/res/layout/activity_main.xml

This is the main file where you will create the registration form interface.

Step 4: Create the registration form layout

Inside activity_main.xml, the registration screen should contain these elements in order:

1. A heading displaying User Registration.
2. A horizontal LinearLayout for the name fields.
3. A First Name input field.
4. A Last Name input field.
5. An Email input field.
6. A Password input field.
7. A Terms and Conditions CheckBox.
8. A Register Button.

Important layout settings:

- The main LinearLayout should use vertical orientation.
- The name fields should be placed inside a horizontal LinearLayout.
- Both name fields should share the available width equally.
- The email and password fields should occupy the available width.

Step 5: Check the XML IDs

When configuring the input fields, use the following IDs:

Element	**ID**
First Name	etFirst
Last Name	etLast
Email	etEmail
Password	etPassword
Terms Checkbox	cbTerms
Register Button	btnRegister

Step 6: Check the folder structure

Your important files should be:

```
RegistrationForm/
└── app/
    └── src/
        └── main/
            ├── java/
            |   └── com/example/lab/
            |       └── MainActivity.java
            ├── res/
            |   └── layout/
            |       └── activity_main.xml
            └── AndroidManifest.xml
```

No additional Java class is required.

Step 7: Save and run

1. Save the XML file.
2. Save the Java file.
3. Click Build → Make Project.
4. Wait for the successful build.
5. Start your Android emulator.
6. Select RegistrationForm in the run configuration.
7. Click Run.
8. Wait for the application to launch.

Step 8: Test the application

1. Check the User Registration heading.
2. Verify that First Name and Last Name appear side by side.
3. Check the Email field.
4. Check the Password field.
5. Verify that the Terms and Conditions checkbox is visible.
6. Check the Register button.
7. Rotate the emulator to landscape.
8. Verify that the registration form is still displayed correctly.
9. Rotate back to portrait.
10. Capture both screenshots.

Expected output

Registration form with two name fields, email, password, checkbox and Register button.

Practical 4: Calculator UI using Grid Layout and Linear Layout

Project: CalculatorUI

Step 1: Create a new project

1. Open Android Studio.
2. Click File → New → New Project.
3. Select Phone and Tablet.
4. Select Empty Views Activity.
5. Click Next.
6. Enter project name: CalculatorUI.
7. Enter package name: com.example.lab.
8. Select Java.
9. Select Minimum SDK API 23 or higher.
10. Choose the save location.
11. Click Finish.
12. Wait for Gradle synchronization.

Step 2: Open MainActivity.java

1. Expand app.
2. Expand java.
3. Expand com.example.lab.
4. Open MainActivity.java.

Location:

app/src/main/java/com/example/lab/MainActivity.java

Keep this file ready for any calculator button functionality you intend to implement.

Step 3: Open activity_main.xml

1. Expand app.
2. Expand res.
3. Expand layout.
4. Open activity_main.xml.
5. Select Code view.

Location:

app/src/main/res/layout/activity_main.xml

Step 4: Create the calculator screen

In the layout editor, arrange the components in this order:

1. Create a vertical LinearLayout as the main container.
2. Add a TextView at the top.
3. Set the TextView as the calculator display.
4. Give the display an initial value of 0.
5. Set the display text size to approximately 36sp.
6. Align the display text to the right.
7. Add a GridLayout below the display.
8. Set the GridLayout column count to 4.
9. Set the GridLayout row count to 4.
10. Set the GridLayout height to occupy the remaining available screen space.

Step 5: Add calculator buttons

Create 16 buttons in the following order.

Row	Column 1	Column 2	Column 3	Column 4
1	7	8	9	/
2	4	5	6	*
3	1	2	3	-
4	C	0	=	+

For every button:

1. Set its text to the corresponding key.
2. Set width and height to share the available space.
3. Set equal column weight.
4. Set equal row weight.
5. Ensure that the buttons fit inside the GridLayout.

Step 6: Check the layout

1. Open the XML Preview.
2. Verify that the display is at the top.
3. Verify that the 16 buttons form four rows and four columns.
4. Check that the screen is not horizontally scrolling.
5. Check that all buttons are visible.

Step 7: Save and run

1. Save the XML file.
2. Save MainActivity.java.
3. Select Build → Make Project.

4. Wait for the successful build.
5. Start your emulator.
6. Select CalculatorUI.
7. Click Run.

Step 8: Test the calculator interface

1. Check that the display appears at the top.
2. Verify all four rows.
3. Count the 16 buttons.
4. Verify the arrangement of numbers and operators.
5. Rotate the emulator if desired.
6. Capture a screenshot.

Important: Your lab manual describes the calculator interface and makes calculation logic optional. For the basic UI practical, verify the layout and buttons. A working calculator requires additional click functionality.

Expected output

A calculator interface with a display and a 4 × 4 button grid.

Practical 5: Externalizing Resources and Toast Message

Project: RegistrationForm

Continue Practical 3

Do not create a new project. Open your existing RegistrationForm project from Practical 3.

Step 1: Open the existing project

1. Open Android Studio.
2. Click File → Open.
3. Browse to the folder where you saved RegistrationForm.
4. Select the RegistrationForm project folder.
5. Click OK.
6. Wait for the project to load.
7. Wait for Gradle synchronization.

Step 2: Open the strings.xml file

1. Open the Project panel.
2. Expand app.
3. Expand res.
4. Expand values.
5. Open strings.xml.

Exact location:

app/src/main/res/values/strings.xml

This file stores the text used throughout your application.

Step 3: Check the strings resources

Create or verify the following resource entries:

Resource name	Display text
app_name	RegistrationForm
title	User Registration
hint_first	First name
hint_last	Last name
btn_register	Register
welcome	Welcome, username!

Keep the resource names consistent because they will be referenced by the XML layout and Java Activity.

Step 4: Open colors.xml

1. Stay inside the res folder.
2. Expand values.
3. Check whether colors.xml exists.
4. If it exists, open it.
5. If it does not exist, right-click values.
6. Select New → Values Resource File.
7. Enter the filename colors.
8. Keep the resource type as Values.
9. Click OK.

Exact location:

app/src/main/res/values/colors.xml

Step 5: Configure colors

Create or verify these three color resources:

Resource name	Color
primary	#1F3A5F
accent	#1B998B
white	#FFFFFF

Step 6: Modify activity_main.xml

1. Expand res.
2. Expand layout.

3. Open activity_main.xml.
4. Select Code view.
5. Locate the registration form elements.
6. Replace the hardcoded heading text with the appropriate string resource.
7. Replace the First Name hint with its string resource.
8. Replace the Last Name hint with its string resource.
9. Replace the Register button text with its string resource.
10. Set the Register button background tint to the primary color resource.
11. Set the button text color to the white color resource.
12. Check that the original registration form elements remain present.

Location:

app/src/main/res/layout/activity_main.xml

Step 7: Modify MainActivity.java

1. Expand java.
2. Expand com.example.lab.
3. Open MainActivity.java.
4. Locate the existing registration Activity implementation.
5. Keep the existing form structure.
6. Add the Register button click functionality.
7. Ensure that the First Name field is accessed.
8. Ensure that the Register button is accessed.
9. Verify that the entered name is checked.
10. Configure the application to display a short warning if the name is empty.
11. Configure the application to display a personalised Welcome Toast when a name is entered.

Location:

app/src/main/java/com/example/lab/MainActivity.java

Step 8: Check the resource references

Verify that the following resource names are consistent across the files:

- hint_first
- hint_last
- btn_register
- welcome
- primary
- accent

- white

Do not create separate resource files for each string or color.

Step 9: Save and run

1. Press Ctrl + S.
2. Click Build → Make Project.
3. Wait for the build to finish.
4. Start your emulator.
5. Select RegistrationForm.
6. Click Run.

Step 10: Test the application

Test A – Empty name

1. Leave the First Name field empty.
2. Click Register.
3. Check that the warning Toast appears.

Test B – Enter a name

1. Enter Rutuja in First Name.
2. Click Register.
3. Verify that the Toast displays Welcome, Rutuja!.

Test C – Resource verification

1. Check the registration heading.
2. Check the input hints.
3. Check the button text.
4. Check the button background color.
5. Verify that the application still works after using the resource files.

Step 11: Capture screenshot

Capture a screenshot showing the registration form and the Welcome Toast.

Expected output

Personalised Welcome Toast after clicking Register.

Practical 6: Two Fragments in Portrait and Landscape

Project: FragmentDemo

Step 1: Create a new project

1. Open Android Studio.
2. Select File → New → New Project.
3. Select Phone and Tablet.

4. Select Empty Views Activity.
5. Click Next.
6. Enter project name: FragmentDemo.
7. Enter package name: com.example.lab.
8. Select Java.
9. Select Minimum SDK API 23 or higher.
10. Select the save location.
11. Click Finish.
12. Wait for Gradle synchronization.

Step 2: Open the Java package

1. Expand app.
2. Expand java.
3. Expand com.example.lab.
4. Locate MainActivity.java.

You will create two additional Java classes inside this package.

Step 3: Create ListFragment1.java

1. Right-click com.example.lab.
2. Select New.
3. Select Java Class.
4. Enter the class name:

ListFragment1

5. Press Enter.
6. Open the newly created Java file.
7. Keep it ready for the list fragment implementation.

Exact location:

app/src/main/java/com/example/lab/ListFragment1.java

Step 4: Create DetailFragment.java

1. Right-click com.example.lab.
2. Select New.
3. Select Java Class.
4. Enter:

DetailFragment

5. Press Enter.
6. Open the newly created file.

Exact location:

app/src/main/java/com/example/lab/DetailFragment.java

Step 5: Create fragment_list.xml

1. Expand app.
2. Expand res.
3. Expand layout.
4. Right-click the layout folder.
5. Select New → Layout Resource File.
6. Enter filename:

fragment_list

7. Select an appropriate root element.
8. Click OK.
9. Open the new XML file.

Exact location:

app/src/main/res/layout/fragment_list.xml

Step 6: Configure the list fragment layout

1. Open fragment_list.xml.
2. Select Code view.
3. Create a ListView in the layout.
4. Set its ID to listView.
5. Set its width to match the available parent width.
6. Set its height to match the available parent height.
7. Save the file.

The list will contain these four items, as specified in your manual:

- Android
- Fragments
- Intents
- SQLite

Step 7: Create fragment_detail.xml

1. Right-click the layout folder.
2. Select New → Layout Resource File.
3. Enter filename:

fragment_detail

4. Click OK.

5. Open the newly created file.
6. Add a TextView as the detail display.
7. Set the TextView ID to tvDetail.
8. Set its width and height according to the layout requirements.
9. Save the file.

Exact location:

app/src/main/res/layout/fragment_detail.xml

Step 8: Create the landscape resource directory

This is an important step.

Android uses a separate layout directory for landscape orientation.

1. Right-click the res folder.
2. Select New.
3. Select Android Resource Directory.
4. In Resource type, select layout.
5. Find Available qualifiers.
6. Select Orientation.
7. Click the arrow to add it.
8. In the Orientation dropdown, select Landscape.
9. Check that the directory name becomes layout-land.
10. Click OK.

You should now see:

app/src/main/res/layout-land/

Android Studio: Creating landscape layouts - Stack Overflow

stackoverflow.com

Step 9: Create landscape activity_main.xml

1. Right-click layout-land.
2. Select New.
3. Select Layout Resource File.
4. Enter filename:

activity_main

5. Click OK.
6. Open the new file.

Exact location:

app/src/main/res/layout-land/activity_main.xml

This is the layout used when the device is in landscape mode.

Step 10: Configure the portrait layout

Open the original activity_main.xml.

Location:

app/src/main/res/layout/activity_main.xml

Configure the screen to contain:

1. A FrameLayout as the main container.
2. The ListFragment1 fragment.
3. The fragment should occupy the available screen.

This is the portrait layout.

Step 11: Configure the landscape layout

Open:

app/src/main/res/layout-land/activity_main.xml

Configure the screen to contain:

1. A horizontal LinearLayout.
2. ListFragment1 on the left.
3. DetailFragment on the right.
4. List fragment width controlled by layout weight 1.
5. Detail fragment width controlled by layout weight 2.
6. Both fragments should occupy the available height.

Step 12: Check the Java and XML file structure

Your project should contain:

```
FragmentDemo/
└── app/
    └── src/
        └── main/
            ├── java/com/example/lab/
            |   ├── MainActivity.java
            |   ├── ListFragment1.java
            |   └── DetailFragment.java
            |
            ├── res/
            |   ├── layout/
            |   |   ├── activity_main.xml
```

```
|   |   ├── fragment_list.xml
|   |   └── fragment_detail.xml
|   |
|   └── layout-land/
|       └── activity_main.xml
|
└── AndroidManifest.xml
```

Step 13: Save and run

1. Save all Java files.
2. Save all XML files.
3. Select Build → Make Project.
4. Wait for the successful build.
5. Start your emulator.
6. Select FragmentDemo.
7. Click Run.
8. Wait for the application to open.

Step 14: Test portrait mode

1. Keep the emulator in portrait orientation.
2. Open the application.
3. Verify that only the list fragment appears.
4. Check that all four list items are visible.

Step 15: Test landscape mode

1. Rotate the emulator to landscape.
2. Wait for the layout to refresh.
3. Verify that the list appears on the left.
4. Verify that the detail fragment appears on the right.
5. Check that both fragments are visible simultaneously.

Step 16: Capture screenshots

Save two screenshots:

- Portrait mode displaying the list.
- Landscape mode displaying the list and detail fragments.

Expected output

Portrait displays one list pane; landscape displays two panes.

Practical 7: Fragment to Activity Communication

Project: FragmentDemo

Continue Practical 6

Do not create another project. This practical extends FragmentDemo.

Step 1: Open Practical 6 project

1. Open Android Studio.
2. Click File → Open.
3. Browse to your FragmentDemo folder.
4. Select the project.
5. Click OK.
6. Wait for Gradle synchronization.

Step 2: Open ListFragment1.java

1. Expand app.
2. Expand java.
3. Expand com.example.lab.
4. Open ListFragment1.java.

Location:

app/src/main/java/com/example/lab/ListFragment1.java

Step 3: Locate the existing list implementation

1. Find the existing list fragment implementation.
2. Keep the existing four list items.
3. Locate the section where the ListView adapter is configured.
4. Prepare this class for the Activity communication interface.

Step 4: Add the communication interface

Inside ListFragment1.java:

1. Create the OnItemSelectedListener interface.
2. Include the item-selection callback declaration.
3. Create a listener variable.
4. Add the listener attachment procedure inside onAttach().
5. Make sure the hosting Activity is checked for interface implementation.
6. Add an item click listener to the ListView.
7. Configure the selected list item to be sent through the listener.

Step 5: Open DetailFragment.java

1. Expand com.example.lab.
2. Open DetailFragment.java.

Location:

app/src/main/java/com/example/lab/DetailFragment.java

Step 6: Configure the detail update

1. Locate the existing DetailFragment class.
2. Find its view creation section.
3. Add a public method named updateText.
4. Configure this method to receive the selected item.
5. Locate the TextView using its existing ID tvDetail.
6. Configure the TextView to display the selected item.
7. Save the file.

Step 7: Open MainActivity.java

1. Expand com.example.lab.
2. Open MainActivity.java.

Location:

app/src/main/java/com/example/lab/MainActivity.java

Step 8: Implement the interface in MainActivity

1. Open the MainActivity class declaration.
2. Make MainActivity implement ListFragment1.OnItemSelectedListener.
3. Locate the existing Activity creation section.
4. Keep the existing layout configuration.
5. Add the required item-selection callback implementation.
6. Locate DetailFragment through the FragmentManager.
7. Check whether DetailFragment is available.
8. Call its updateText() method with the selected item.
9. Save the file.

Step 9: Check the landscape and portrait layouts

Do not delete either layout.

Verify these files:

Portrait:

app/src/main/res/layout/activity_main.xml

Landscape:

app/src/main/res/layout-land/activity_main.xml

Check that the landscape detail fragment still has the ID detailFragment.

Step 10: Save and build

1. Save all three Java files.
2. Open Build.
3. Select Make Project.
4. Wait for compilation.
5. Check for errors.
6. Correct any missing class or ID references.

Step 11: Run the application

1. Start the emulator.
2. Select FragmentDemo.
3. Click Run.
4. Wait for the application to open.

Step 12: Test in landscape

1. Rotate the emulator to landscape.
2. Check that the list is displayed on the left.
3. Check that the detail area is displayed on the right.
4. Tap Android.
5. Verify that the detail area displays the selected item.
6. Tap Fragments.
7. Verify that the detail text changes.
8. Tap Intents.
9. Verify that the detail text changes.
10. Tap SQLite.
11. Verify that the detail text changes.

The expected detail message format is You selected: [item].

Step 13: Capture screenshot

Capture the landscape screen after selecting one of the items.

Expected output

Selecting a list item updates the detail fragment through the hosting Activity.

Practical 8: Explicit and Implicit Intents

Project: IntentDemo

Step 1: Create a new project

1. Open Android Studio.
2. Click File → New → New Project.
3. Select Phone and Tablet.

4. Select Empty Views Activity.
5. Click Next.
6. Enter project name: IntentDemo.
7. Enter package name: com.example.lab.
8. Select Java.
9. Select Minimum SDK API 23 or higher.
10. Select the save location.
11. Click Finish.
12. Wait for Gradle synchronization.

Step 2: Open MainActivity.java

1. Expand app.
2. Expand java.
3. Expand com.example.lab.
4. Open MainActivity.java.

Location:

app/src/main/java/com/example/lab/MainActivity.java

Step 3: Open activity_main.xml

1. Expand app.
2. Expand res.
3. Expand layout.
4. Open activity_main.xml.
5. Select Code view.

Location:

app/src/main/res/layout/activity_main.xml

Step 4: Configure the first Activity screen

The first screen must contain:

1. A TextView displaying a heading.
2. A Button for navigating to the second Activity.
3. Set the button ID to btnNext.
4. Make sure the button is visible.

Step 5: Create SecondActivity.java

1. Right-click com.example.lab.
2. Select New.
3. Select Java Class.

4. Enter the name:

SecondActivity

5. Press Enter.
6. Open the newly created Java file.

Location:

app/src/main/java/com/example/lab/SecondActivity.java

Step 6: Create activity_second.xml

1. Expand res.
2. Expand layout.
3. Right-click layout.
4. Select New → Layout Resource File.
5. Enter filename:

activity_second

6. Click OK.
7. Open the newly created XML file.

Location:

app/src/main/res/layout/activity_second.xml

Step 7: Configure the second Activity screen

The second screen must contain:

1. A TextView to display the message received from MainActivity.
2. Set the TextView ID to tvMessage.
3. Add a Button for opening a web page.
4. Set the button ID to btnOpen.
5. Ensure that both elements are visible.

Step 8: Configure AndroidManifest.xml

1. Expand manifests.
2. Open AndroidManifest.xml.
3. Find the application section.
4. Check whether SecondActivity has been registered.
5. If it is not registered, add an Activity declaration for SecondActivity inside the application section.
6. Set its exported property to false.
7. Keep MainActivity as the launcher Activity.
8. Save the manifest.

Location:

app/src/main/AndroidManifest.xml

Step 9: Configure the explicit Intent

Open:

app/src/main/java/com/example/lab/MainActivity.java

1. Locate the Next button.
2. Configure its click action.
3. Select SecondActivity as the destination.
4. Configure the message to be passed as an Intent extra.
5. Use the message From Main Activity.
6. Ensure that clicking Next opens SecondActivity.

Step 10: Configure the second Activity

Open:

app/src/main/java/com/example/lab/SecondActivity.java

1. Connect the second Activity to activity_second.xml.
2. Locate the TextView using ID tvMessage.
3. Configure it to receive the message from the first Activity.
4. Locate the Open button using ID btnOpen.
5. Configure the button to open a web page through an implicit Intent.
6. Use the Google website URL specified in the manual.
7. Save the file.

Step 11: Check your project structure

```
IntentDemo/
└── app/
    └── src/
        └── main/
            ├── java/com/example/lab/
            |   ├── MainActivity.java
            |   └── SecondActivity.java
            |
            ├── res/layout/
            |   ├── activity_main.xml
            |   └── activity_second.xml
            |
            └── AndroidManifest.xml
```

Step 12: Save and build

1. Save all files.
2. Click Build → Make Project.
3. Wait for the build to complete.
4. Check for errors.
5. Start the emulator.

Step 13: Run and test

Test A – Explicit Intent

1. Open IntentDemo.
2. Verify that the first screen appears.
3. Click the Next button.
4. Verify that SecondActivity opens.
5. Check that the received message displays From Main Activity.

Test B – Implicit Intent

1. Stay on the second screen.
2. Click the Open button.
3. If Android displays an application chooser, select a browser.
4. Verify that the browser opens.
5. Check that Google loads.

Step 14: Capture screenshots

Capture:

- First Activity screen.
- Second Activity displaying the received message.
- Browser displaying the opened website.

Expected output

SecondActivity displays the received message and the Open button launches the browser.

Practical 9: SQLite Database – Create and Insert

Project: StudentDB

Important: Your uploaded manual specifies the project name StudentDB. Use this name instead of SQLiteDemo.

Step 1: Create a new project

1. Open Android Studio.
2. Click File → New → New Project.
3. Select Phone and Tablet.
4. Select Empty Views Activity.

5. Click Next.
6. Enter project name: StudentDB.
7. Enter package name: com.example.lab.
8. Select Java.
9. Select Minimum SDK API 23 or higher.
10. Choose the project save location.
11. Click Finish.
12. Wait for Gradle synchronization.

Step 2: Open the main Java package

1. Expand app.
2. Expand java.
3. Expand com.example.lab.
4. Locate MainActivity.java.

Step 3: Create DBHelper.java

1. Right-click com.example.lab.
2. Select New.
3. Select Java Class.
4. Enter class name:

DBHelper

5. Press Enter.
6. Open the newly created Java file.

Exact location:

app/src/main/java/com/example/lab/DBHelper.java

This is the database management file.

Step 4: Open MainActivity.java

1. Expand com.example.lab.
2. Double-click MainActivity.java.
3. Keep it ready for the database insertion functionality.

Exact location:

app/src/main/java/com/example/lab/MainActivity.java

Step 5: Open activity_main.xml

1. Expand app.
2. Expand res.
3. Expand layout.

4. Open activity_main.xml.
5. Select Code view.

Exact location:

app/src/main/res/layout/activity_main.xml

Step 6: Create the student registration interface

Inside activity_main.xml, arrange these components:

1. Add a heading for Student Database.
2. Add an input field for Student Name.
3. Add an input field for Roll Number.
4. Add a Save button.

Step 7: Set the required XML IDs

Check that the following IDs are assigned correctly:

UI component	ID
Student Name	etName
Roll Number	etRoll
Save Button	btnSave

For the Roll Number field:

1. Open its properties.
2. Find the input type.
3. Select Number.
4. Check that the keyboard accepts numeric input.

Step 8: Configure DBHelper.java

Open:

app/src/main/java/com/example/lab/DBHelper.java

Inside this class, the database configuration must include:

Database setting	Value
Database name	college.db
Database version	1
Table name	students
Primary key	id
Name column	name
Roll column	roll

The table must contain:

- id: Integer primary key with auto-increment.
- name: Text and not null.
- roll: Integer and not null.

The database helper should also handle:

1. Database creation.
2. Table creation.
3. Database upgrade.
4. Student record insertion.

Step 9: Configure MainActivity.java

Open:

app/src/main/java/com/example/lab/MainActivity.java

Follow these steps:

1. Connect MainActivity to activity_main.xml.
2. Initialize the DBHelper object.
3. Locate the Student Name field.
4. Locate the Roll Number field.
5. Locate the Save button.
6. Configure the Save button click event.
7. Read the Student Name entered by the user.
8. Read the Roll Number entered by the user.
9. Check whether both fields are empty.
10. If either field is empty, display a warning message.
11. If both fields are entered, send the details to DBHelper.
12. Insert the student record into the database.
13. Display a success message if the record is saved.
14. Display an error message if insertion fails.
15. Clear the input fields after successful insertion.

Step 10: Check the project structure

StudentDB/

└── app/

  └── src/

    └── main/

      ├── java/com/example/lab/

      | ├── MainActivity.java

```
|   └── DBHelper.java
|
├── res/layout/
|   └── activity_main.xml
|
└── AndroidManifest.xml
```

Step 11: Save and build

1. Save MainActivity.java.
2. Save DBHelper.java.
3. Save activity_main.xml.
4. Click Build.
5. Select Make Project.
6. Wait for successful compilation.
7. Check that no unresolved resource or class errors remain.

Step 12: Run the application

1. Open Device Manager.
2. Start your Android emulator.
3. Select StudentDB.
4. Click Run.
5. Wait for the application to open.

Step 13: Test database insertion

Test A – Empty fields

1. Leave both fields empty.
2. Click Save.
3. Verify that a warning appears.

Test B – Enter student details

1. Enter a student name, for example Anjali Narkhede.
2. Enter roll number 12.
3. Click Save.
4. Verify that the message Student saved appears.
5. Check that both input fields are cleared.

Test C – Insert another student

1. Enter another student name.
2. Enter another roll number.

3. Click Save.
4. Verify that the second record is also saved.

Step 14: Verify the database

1. Keep StudentDB running on the emulator.
2. Open the Device Explorer tool window in Android Studio.
3. Select the running emulator.
4. Navigate to the application's private data directory.
5. Find the database directory.
6. Look for college.db.

Depending on Android Studio version, you may also use App Inspection → Database Inspector.

Use SQL to read and write to a database | Android Developers

developer.android.com

Step 15: Capture screenshot

Capture:

- Student registration form.
- Student saved Toast message.
- Database verification screen, if available.

Expected output

Student information is inserted into the SQLite database and the success message is displayed.

Practical 10: SQLite Database – Retrieve with Cursor and ListView

Project: StudentDB

Continue Practical 9

Do not create a new project or a new database. Continue with the StudentDB project.

Step 1: Open the Practical 9 project

1. Open Android Studio.
2. Click File → Open.
3. Browse to your StudentDB project folder.
4. Select StudentDB.
5. Click OK.
6. Wait for Gradle synchronization.

Step 2: Open DBHelper.java

1. Expand app.
2. Expand java.
3. Expand com.example.lab.

4. Open DBHelper.java.

Location:

app/src/main/java/com/example/lab/DBHelper.java

Step 3: Check existing database configuration

Before making changes, verify that the existing database configuration is still present.

Check:

- Database name: college.db.
- Database version: 1.
- Table name: students.
- Student ID column.
- Student name column.
- Student roll number column.

Do not create another DBHelper.java file.

Step 4: Add the database retrieval functionality

Inside DBHelper.java:

1. Open the existing database helper class.
2. Locate the existing insertion functionality.
3. Keep the existing database creation method.
4. Keep the existing table creation configuration.
5. Add the required database retrieval functionality.
6. Make sure the database is opened for reading.
7. Retrieve student records from the students table.
8. Retrieve the ID, name and roll number.
9. Organize the retrieved records by roll number.
10. Move through each returned database record.
11. Store the retrieved student information in a collection.
12. Close the Cursor after reading all records.
13. Make the retrieved student collection available to MainActivity.

Step 5: Check imports in DBHelper.java

Ensure the required Java and Android database imports are available.

In particular, check for:

- Cursor.
- SQLiteDatabase.
- ArrayList.

- List.

Do not remove existing imports required by Practical 9.

Step 6: Open activity_main.xml

1. Expand app.
2. Expand res.
3. Expand layout.
4. Open activity_main.xml.
5. Select Code view.

Location:

app/src/main/res/layout/activity_main.xml

Step 7: Add the ListView

1. Locate the existing Student Name field.
2. Locate the existing Roll Number field.
3. Locate the existing Save button.
4. Keep all three components.
5. Add a ListView below the Save button.
6. Set the ListView ID to lvStudents.
7. Set its width to match the parent.
8. Give it sufficient height to display the records.

Step 8: Add the Show button

1. Add another Button below the existing Save button.
2. Set its text to Show Students.
3. Set its ID to btnShow.
4. Check that it is visible.
5. Save activity_main.xml.

The screen should now contain:

1. Student Name input field.
2. Roll Number input field.
3. Save button.
4. Show Students button.
5. ListView.

Step 9: Open MainActivity.java

1. Expand java.
2. Expand com.example.lab.

3. Open MainActivity.java.

Location:

app/src/main/java/com/example/lab/MainActivity.java

Step 10: Preserve Practical 9 functionality

This step is very important.

1. Do not delete the existing database initialization.
2. Do not delete the DBHelper object initialization.
3. Do not delete the Student Name field.
4. Do not delete the Roll Number field.
5. Do not delete the Save button.
6. Keep the existing student insertion functionality.
7. Continue modifying the same MainActivity.java file.

Step 11: Add ListView functionality

Inside MainActivity.java:

1. Locate the existing onCreate() section.
2. Locate the ListView using the ID lvStudents.
3. Locate the Show button using the ID btnShow.
4. Configure the Show button click event.
5. Call the student retrieval method from DBHelper.
6. Receive the list of student records.
7. Check whether the list is empty.
8. If there are no records, display a Toast message saying No records found.
9. If records exist, display them in the ListView.
10. Use an ArrayAdapter to connect the student collection to the ListView.
11. Save the file.

Step 12: Check your final project structure

StudentDB/

└── app/

  └── src/

    └── main/

      ├── java/

      | └── com/example/lab/

      | ├── MainActivity.java

      | └── DBHelper.java

```
|
├── res/
|   └── layout/
|       └── activity_main.xml
|
└── AndroidManifest.xml
```

No additional Java class or database file needs to be created.

Step 13: Save and build

1. Press Ctrl + S.
2. Click Build.
3. Select Make Project.
4. Wait for the build to finish.
5. Check for any missing IDs.
6. Check that the ListView is correctly referenced.
7. Check that the Show button is correctly referenced.
8. Verify that the existing DBHelper class is recognized.

Step 14: Run the application

1. Open Device Manager.
2. Start your emulator.
3. Select StudentDB.
4. Click Run.
5. Wait for the application to open.

Step 15: Test student insertion

1. Enter a student name.
2. Enter a roll number.
3. Click Save.
4. Verify the Student saved message.
5. Enter another student's details.
6. Click Save again.
7. Verify that multiple records can be inserted.

Step 16: Test student retrieval

1. Click the Show Students button.
2. Wait for the ListView to refresh.
3. Check whether the saved student records appear.

4. Verify that student roll numbers are displayed.
5. Verify that student names are displayed.
6. Verify that student IDs are displayed.
7. Check that records are arranged by roll number.

For example, the list should display a record in this format:

Roll 12 - Anjali Narkhede (id 1)

Step 17: Test empty database condition

1. If using a fresh application installation, do not insert any students.
2. Open the application.
3. Click Show Students.
4. Verify that the No records found message appears.

Note: If you already inserted records in Practical 9, they will remain in the database. To test the empty state, use a fresh installation or clear the application data. Clearing data deletes the saved student records.

Step 18: Capture screenshots

Capture:

1. Student registration screen.
2. Screen showing saved student records in ListView.
3. Screen showing the empty database message, if tested separately.